

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
Fakulta elektrotechniky a informatiky

JPA a REST

Persistencia a webové služby v Jave

Typ dokumentu:Technická dokumentácia / Študijné poznámky

Predmet:Vývoj softvérových aplikácií

Platforma:Jakarta EE 11, EclipseLink 4.x, GlassFish 8

Akademický rok:2024/2025

Verzia:1.0

Dátum vydania:Máj 2025

Obsah

Abstrakt	4
1 Úvod do JPA	5
1.1 Motivácia – impedančný nesúlad	5
1.2 Základné pojmy	5
1.3 Minimálna entitná trieda	5
1.4 Konfigurácia persistence.xml	6
1.5 Základné CRUD operácie	6
2 Entitné triedy a JPA anotácie	8
2.1 Mapovanie dátových typov	8
2.2 Generovanie primárnych kľúčov	8
2.3 Konfigurácia stĺpcov	8
2.4 Dotazovací jazyk JPQL	9
3 Asociácie medzi entitami	10
3.1 Jednosmerné asociácie	10
3.1.1 N:1 asociácia – @ManyToOne	10
3.1.2 1:N asociácia – @OneToMany + @JoinColumn	10
3.1.3 N:N asociácia – @ManyToMany	10
3.2 Obojsmerné asociácie a parameter mappedBy	10
3.3 Stratégia načítavania – FetchType	11
4 Špecifické situácie pri mapovaní	12
4.1 Kompozícia – CascadeType a orphanRemoval	12
4.2 Asociácia 1:1 – @OneToOne	12
4.3 Vložená trieda – @Embedded / @Embeddable	12

4.4 Sekundárna tabuľka – @SecondaryTable	13
4.5 Zložené primárne kľúče – @EmbeddedId	13
5 Mapovanie hierarchie tried	14
5.1 SINGLE_TABLE	14
5.2 JOINED	14
5.3 TABLE_PER_CLASS	14
5.4 Porovnanie stratégií	15
5.5 Abstraktná nadtrieda – @MappedSuperclass	15
6 REST webové služby	16
6.1 Základné princípy	16
6.2 HTTP metódy a CRUD	16
6.3 Anotácie JAX-RS	16
6.4 Implementácia triedy zdroja	17
6.5 Parametrizácia URI a trieda Response	17
7 Integrácia REST a JPA	19
7.1 Konfigurácia JPA pre webovú aplikáciu	19
7.2 Serializácia do XML/JSON pomocou JAXB	19
7.3 REST klient v Jave	20
8 HATEOAS a pokročilé témy	21
8.1 Problém cyklických referencií	21
8.2 Princíp HATEOAS	21
8.3 Klient so spracovaním HATEOAS linkov	22
9 Záver	23
10 Zoznam skratiek	24

Abstrakt

Tento dokument tvorí komplexnú technickú dokumentáciu k technológiám **Java Persistence API (JPA)** a **REST webovým službám**. Pokrýva celý rozsah od základného objektovo-relačného mapovania (ORM) cez pokročilé techniky konfigurácie entitných tried a asociácií, stratégie mapovania hierarchií tried, až po návrh a implementáciu RESTful webových služieb integrovaných s perzistentnou vrstvou.

Dokumentácia je určená vývojárom Java EE / Jakarta EE aplikácií, ktorí potrebujú prehľadný a systematický referenčný materiál pokrývajúci štandard JPA 3.x a JAX-RS 3.x. Všetky príklady kódu sú napísané v jazyku Java 17+ a overené s implementáciou EclipseLink 4.0 na aplikačnom serveri Eclipse GlassFish 8.

Poznámka: *Kľúčové slová: JPA, EclipseLink, ORM, Entity Manager, JPQL, REST, JAX-RS, Jakarta EE, GlassFish, JAXB, JSON, XML, HATEOAS.*

1 Úvod do JPA

Java Persistence API (JPA) je štandard platformy Jakarta EE pre **objektovo-relačné mapovanie** (ORM). Umožňuje vývojárom pracovať s relačnou databázou prostredníctvom bežných Java objektov bez nutnosti priamo písať SQL príkazy. Referenčnou implementáciou je **EclipseLink**; rozšírenou alternatívou je **Hibernate**.

1.1 Motivácia - impedančný nesúlad

Relačné databázy a objektovo-orientované jazyky vychádzajú z odlišných paradigiem. Tento **impedančný nesúlad** sa prejavuje v týchto oblastiach:

- Identita: OO rozlišuje referenčnú a hodnotovú rovnosť; relačný model používa primárny kľúč.
- Asociácie: OO modeluje väzby ako referencie na objekty; relačný model ich vyjadruje cudzími kľúčmi.
- Dedičnosť: OO podporuje hierarchiu tried; relačný model nemá priamy ekvivalent.
- Granularita: jedna entitná trieda môže zodpovedať viacerým tabuľkám a naopak.

1.2 Základné pojmy

Pojem	Definícia
Persistence Unit	Pomenovaná konfigurácia obsahujúca parametre pripojenia k databáze, zoznam entitných tried a nastavenia JPA. Definovaná v persistence.xml.
Entity class	Java trieda anotovaná @Entity, ktorej inštancie zodpovedajú riadkom databázovej tabuľky. Musí spĺňať konvencie JavaBean.
Entity Manager	Centrálny komponent JPA. Synchronizuje stav entitných objektov s databázou a spravuje Persistence Context.
Persistence Context	Množina entitných objektov spravovaných jedným Entity Managerom. Funguje ako cache prvej úrovne – každá entita je zastúpená najvyšš jednou inštanciou.
Managed object	Entitný objekt prepojený s Persistence Contextom. Zmeny jeho stavu sa automaticky prepisujú do databázy pri commit.
Detached object	Entitný objekt odpojený od Persistence Contextu. Zmeny jeho stavu sa do databázy nezapisujú.

Tabuľka 1.1 – Základné pojmy JPA

1.3 Minimálna entitná trieda

Entitná trieda musí spĺňať tieto požiadavky špecifikácie JPA: anotáciu **@Entity** na úrovni triedy, anotáciu **@Id** pre atribút primárneho kľúča, verejný alebo chránený konštruktor bez parametrov a prístupové metódy getter/setter pre každý perzistovaný atribút.

```
@Entity // mapuje triedu na tabuľku KNIHA
public class Kniha implements Serializable {

    @Id // primárny kľúč
    private String nazov;

    private double cena;

    public Kniha() {} // povinný konštruktor bez parametrov

    public String getNazov() { return nazov; }
    public void setNazov(String n) { this.nazov = n; }
    public double getCena() { return cena; }
    public void setCena(double c) { this.cena = c; }
}
```

Poznámka: Ak sa meno triedy zhoduje s menom tabuľky (case-insensitive) a mená atribútov zodpovedajú názvom stĺpcov, JPA vykoná mapovanie automaticky bez ďalšej konfigurácie.

1.4 Konfigurácia persistence.xml

```
<persistence-unit name="vsaPU" transaction-type="RESOURCE_LOCAL">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <class>sk.example.Kniha</class>

  <properties>
    <!-- JDBC parametre pripojenia -->
    <property name="jakarta.persistence.jdbc.url"
      value="jdbc:postgresql://localhost:5432/edudb"/>
    <property name="jakarta.persistence.jdbc.user" value="edu"/>
    <property name="jakarta.persistence.jdbc.password" value="edu4edu"/>
    <property name="jakarta.persistence.jdbc.driver"
      value="org.postgresql.Driver"/>

    <!-- Správa schémy: none | create | drop-and-create -->
    <property name="jakarta.persistence.schema-generation.database.action"
      value="create"/>
  </properties>
</persistence-unit>
```

1.5 Základné CRUD operácie

```

EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("vsaPU");
EntityManager em = emf.createEntityManager();

// --- čítanie podľa primárneho kľúča ---
Kniha k = em.find(Kniha.class, "Pipi"); // null ak nenájde

// --- vloženie nového záznamu ---
em.getTransaction().begin();
em.persist(novaKniha); // INSERT
em.getTransaction().commit();

// --- modifikácia (managed objekt -> automatický UPDATE) ---
em.getTransaction().begin();
k.setCena(k.getCena() * 0.9);
em.getTransaction().commit();

// --- odstránenie ---
em.getTransaction().begin();
em.remove(k); // DELETE
em.getTransaction().commit();

```

Metóda	Popis
<code>persist(obj)</code>	Vloží nový záznam; objekt sa stane managed
<code>find(Class, id)</code>	Načíta záznam podľa kľúča; vráti null ak neexistuje
<code>merge(obj)</code>	Aktualizuje DB stavom objektu (nemusí byť managed); vráti nový managed objekt
<code>remove(obj)</code>	Odstráni záznam z databázy
<code>detach(obj)</code>	Odpojí objekt od Persistence Contextu
<code>clear()</code>	Odpojí všetky spravované objekty
<code>refresh(obj)</code>	Obnoví stav objektu z DB; zahodí lokálne zmeny

Tabuľka 1.2 – Metódy rozhrania *EntityManager*

2 Entitné triedy a JPA anotácie

Táto kapitola popisuje pokročilú konfiguráciu entitných tried prostredníctvom JPA anotácií: mapovanie dátových typov, mechanizmy generovania kľúčov, správu identity a dotazovací jazyk JPQL.

2.1 Mapovanie dátových typov

```
@Entity
public class Person implements Serializable {

    @Id
    @GeneratedValue // kľúč generuje databáza
    private Long id;

    private String name;
    private double salary;
    private boolean married; // getter: isMarried() – prefix is!

    @Temporal(TemporalType.DATE) // povinné pre java.util.Date
    private Date born; // alternatíva: java.time.LocalDate

    @Enumerated(EnumType.STRING) // uložiť meno konštanty, nie ordinál
    private Stav stav;
}
```

2.2 Generovanie primárnych kľúčov

Stratégia	Popis	Typická DB
AUTO (default)	JPA vyberie vhodnú stratégiu automaticky	Akákoľvek
IDENTITY	Stĺpec AUTO_INCREMENT / SERIAL	MySQL, PostgreSQL
SEQUENCE	Databázová sekvencia	PostgreSQL, Oracle
TABLE	Špeciálna tabuľka pre generovanie kľúčov	Akákoľvek

Tabuľka 2.1 – Stratégie generovania primárnych kľúčov

2.3 Konfigurácia stĺpcov


```

@Entity
@Table(name = "OSOBA") // meno tabuľky != meno triedy
public class Person {

    @Column(name = "MENO", // vlastné meno stĺpca
            nullable = false, // NOT NULL constraint
            length = 100) // VARCHAR(100)
    private String name;

    @Column(unique = true) // UNIQUE constraint
    private String email;

    @Transient // atribút sa NEmapruje na žiadny stĺpec
    private String tempData;
}

```

2.4 Dotazovací jazyk JPQL

JPQL (Jakarta Persistence Query Language) používa namiesto názvov tabuliek a stĺpcov **mená entitných tried a ich atribútov**. Tým zaručuje nezávislosť od konkrétnej databázovej schémy.

```

// --- pomenovaný dotaz definovaný na triede ---
@NamedQuery(
    name = "Person.findBySalary",
    query = "SELECT p FROM Person p WHERE p.salary < :limit"
)
@Entity public class Person { ... }

// --- použitie pomenovaného dotazu ---
TypedQuery<Person> q =
    em.createNamedQuery("Person.findBySalary", Person.class);
q.setParameter("limit", 1500.0);
List<Person> result = q.getResultList();

// --- ad-hoc dotaz ---
TypedQuery<Person> q2 = em.createQuery(
    "SELECT p FROM Person p WHERE p.born IS NULL",
    Person.class);

// --- agregácia ---
TypedQuery<Double> q3 = em.createQuery(
    "SELECT AVG(p.salary) FROM Person p",
    Double.class);
Double priemer = q3.getSingleResult();

```

3 Asociácie medzi entitami

JPA poskytuje komplexnú podporu pre mapovanie všetkých typov asociácií: 1:1, N:1, 1:N a N:N. Asociácie môžu byť jednosmerné alebo obojsmerné.

3.1 Jednosmerné asociácie

3.1.1 N:1 asociácia - @ManyToOne

```
@Entity
public class Kniha {
    @Id private Long id;

    @ManyToOne // FK stĺpec VYDAVATEL_ID v KNIHA
    @JoinColumn(name = "VYDAVATEL_ID",
                nullable = false) // povinný cudzí kľúč
    private Firma vydavatel;
}
```

3.1.2 1:N asociácia - @OneToMany + @JoinColumn

Bez @JoinColumn by JPA vytvoril prepojavacu tabuľku (nevie rozlíšiť 1:N od N:N). Pre priame mapovanie FK do tabuľky asociovanej entity:

```
@Entity
public class Firma {
    @Id private Long id;

    @OneToMany // bez prepojovacej tabuľky
    @JoinColumn(name = "FIRMA_ID") // FK stĺpec priamo v tabuľke KNIHA
    private List<Kniha> publikacie;
}
```

3.1.3 N:N asociácia - @ManyToMany

```
@Entity
public class Kniha {
    @Id private Long id;

    @ManyToMany
    @JoinTable(
        name = "KNIHA_AUTOR",
        joinColumns = @JoinColumn(name = "KNIHA_FK"),
        inverseJoinColumns = @JoinColumn(name = "AUTOR_FK")
    )
    private List<Osoba> autori;
}
```

Poznámka: Metódu `persist()` treba zavolať pre každý nový asociovaný objekt v rámci tej istej transakcie.

3.2 Obojsmerné asociácie a parameter `mappedBy`

Obojsmerná asociácia pozostáva z dvoch navzájom inverzných jednosmerných. Vlastník asociácie (owner) obsahuje cudzí kľúč v databáze. Ne-vlastník je označený parametrom **mappedBy**:

```
// --- owner – obsahuje FK stĺpec v DB ---
@Entity
public class Kniha {
    @ManyToOne
    private Firma vydavatel; // VYDAVATEL_ID je FK v tabuľke KNIHA
}

// --- inverzná strana – mappedBy = meno atribútu ownera ---
@Entity
public class Firma {
    @OneToMany(mappedBy = "vydavatel") // bez JoinColumn
    private List<Kniha> publikacie;
}
```

Poznámka: Iba owner strana riadi zápis FK do databázy. Nastavenie referencie výhradne na non-owner strane nebude mať žiadny efekt v databáze.

3.3 Stratégia načítavania - FetchType

FetchType	Správanie	Predvolené pre
EAGER	Asociované objekty načítané okamžite spolu s find()	@ManyToOne, @OneToOne
LAZY	Asociované objekty načítané až pri prvom prístupe	@OneToMany, @ManyToMany

Tabuľka 3.1 – Stratégie načítavania asociácií

```
@OneToMany(fetch = FetchType.EAGER) // zmena predvoleného správania
@ManyToOne(fetch = FetchType.LAZY) // zmena predvoleného správania
```

Poznámka: EAGER pri @ManyToMany môže spôsobiť kaskádové načítanie celého grafu referencií a výrazne degradovať výkon.

4 Špecifické situácie pri mapovaní

4.1 Kompozícia - CascadeType a orphanRemoval

```
@Entity
public class Kosik {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(
        cascade = CascadeType.REMOVE, // remove(kosik) -> zmaže aj položky
        orphanRemoval = true // odobratie zo zoznamu -> DELETE
    )
    @JoinColumn(name = "KOSIK_FK")
    private List<Polozka> polozky;
}
```

4.2 Asociácia 1:1 - @OneToOne

```
@Entity
public class Poistenie {
    @Id private Long cislo;

    @OneToOne // owner strana
    @JoinColumn(nullable = false, // NOT NULL
        unique = true) // UNIQUE – jedno auto = jedno poistenie
    private Auto vozidlo;
}

@Entity
public class Auto {
    @Id private String spz;

    @OneToOne(mappedBy = "vozidlo") // inverzná strana
    private Poistenie pzp;
}
```

4.3 Vložená trieda - @Embedded / @Embeddable

```
@Embeddable // nie je entita, nemá vlastnú tabuľku
public class Adresa {
    private String ulica;
    private String mesto;
    private String psc;
}

@Entity
public class Osoba {
    @Id private Long id;

    @Embedded // stĺpce ULICA, MESTO, PSC budú v OSOBA
    private Adresa adresa;
}
```

4.4 Sekundárna tabuľka - @SecondaryTable

```
@Entity
@Table(name = "OSOBA")
@SecondaryTable(name = "KONTAKT") // zdieľa PK s OSOBA
public class Osoba {
    @Id private Long id;

    @Column(name = "MENO") // v primárnej tabuľke OSOBA
    private String name;

    @Column(table = "KONTAKT") // v sekundárnej tabuľke KONTAKT
    private String email;

    @Column(table = "KONTAKT",
            name = "MOBIL")
    private String telefon;
}
```

4.5 Zložené primárne kľúče - @EmbeddedId

```
@Embeddable
public class ZaznamPK implements Serializable {
    private String pacient;

    @Temporal(TemporalType.DATE)
    private Date datum;

    // povinné: equals() a hashCode()
}

@Entity
public class Zaznam {
    @EmbeddedId
    private ZaznamPK pk; // namiesto @Id

    private double teplota;
    private double pulz;
}
```

5 Mapovanie hierarchie tried

JPA definuje tri stratégie mapovania dedičnosti tried do relačnej databázy, určené anotáciou **@Inheritance(strategy = InheritanceType.X)** na koreňovej triede.

5.1 SINGLE_TABLE

Všetky triedy hierarchie sa mapujú do jednej tabuľky. Diskriminátor DTYPE identifikuje konkrétnu triedu záznamu.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public class Element {
    @Id private Long id;
    private String name;
    private int xpos, ypos;
}

@Entity // stĺpce text, fontSize, font v tabuľke ELEMENT
public class TextElement extends Element {
    private String text;
    private int fontSize;
    private String font;
}
```

- **Výhoda:** jednoduchá štruktúra, rýchle dotazy bez JOIN.
- **Nevýhoda:** stĺpce podtried sú pre záznamy ostatných podtried prázdne (NULL).

5.2 JOINED

Každá trieda má vlastnú tabuľku len so svojimi atribútmi. PK podtriedy je zároveň FK do tabuľky rodiča.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Element { ... } // tabuľka ELEMENT: id, dtype, name, xpos, ypos

@Entity // tabuľka TEXTELEMENT: id(FK), text, fontSize
public class TextElement extends Element {
    private String text;
    private int fontSize;
}
```

- **Výhoda:** normalizovaná databáza, žiadne NULL hodnoty.
- **Nevýhoda:** každý dotaz vyžaduje JOIN viacerých tabuliek.

5.3 TABLE_PER_CLASS

Každá konkrétna trieda má vlastnú tabuľku so všetkými atribútmi vrátane zdedených.

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Element { ... } // tabuľka ELEMENT: id, name, xpos, ypos

@Entity // TEXTELEMENT: id, name, xpos, ypos, text, fontSize, font
public class TextElement extends Element { ... }
```

- **Výhoda:** rýchle dotazy na konkrétnu triedu.
- **Nevýhoda:** dotazy na celú hierarchiu vyžadujú UNION.

5.4 Porovnanie stratégií

Kritérium	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Počet tabuliek	1	1 na triedu	1 na konkrétnu triedu
NULL hodnoty	Veľa	Žiadne	Žiadne
find() – trieda OK	Rýchle	JOIN (1 dotaz)	Rýchle (1 dotaz)
find() – bez triedy	Rýchle (DTYPE)	Rýchle (DTYPE)	UNION – viac dotazov
JPQL na hierarchiu	Rýchle	JOIN	UNION (pomalé)
Odporúčanie	Málo podtried	Zložitá hierarchia	Nezávislé podtriedy

Tabuľka 5.1 – Porovnanie stratégií mapovania hierarchie tried

5.5 Abstraktná nadtrieda - @MappedSuperclass

```
@MappedSuperclass // žiadna vlastná tabuľka
public abstract class Element implements Serializable {
    @Id @GeneratedValue
    private Long id;
    private String name; // atribúty sa presunú do tabuliek podtried
}
```

6 REST webové služby

Representational State Transfer (REST) je architektonický štýl navrhnutý pre distribuované systémy. Definuje sadu obmedzení, ktorých dodržaním vzniká škálovateľné, bezstavové a jednotné komunikačné rozhranie. Knížnica **JAX-RS** (Jakarta RESTful Web Services) poskytuje sadu anotácií pre implementáciu REST služieb v Jave.

6.1 Základné princípy

- **Klient-server:** klient a server sú striktne oddelené.
- **Bezstavovosť:** každá požiadavka obsahuje všetky potrebné informácie; server neuchováva stav klienta.
- **Jednotné rozhranie:** zdroje manipulované jednotnou sadou HTTP metód.
- **Zdroj (resource):** ľubovoľný objekt identifikovaný URI; reprezentovaný vo formáte XML, JSON, text a iných.

6.2 HTTP metódy a CRUD

HTTP	CRUD	Idempotentná	Typické použitie
GET	Read	Áno	Načítanie zdroja alebo kolekcie
POST	Create	Nie	Nový zdroj; URI určuje server
PUT	Update/Replace	Áno	Aktualizácia; URI určuje klient
DELETE	Delete	Áno	Odstránenie zdroja
PATCH	Partial Update	Nie	Čiastočná aktualizácia atribútov

Tabuľka 6.1 – HTTP metódy a zodpovedajúce CRUD operácie

6.3 Anotácie JAX-RS

Anotácia	Popis
@Path("uri")	URI triedy alebo metódy zdroja
@GET / @POST / @PUT / @DELETE	Mapovanie HTTP metódy na Java metódu
@Produces(MediaType.X)	Formát odpovede: TEXT_PLAIN, APPLICATION_XML, APPLICATION_JSON
@Consumes(MediaType.X)	Akceptovaný formát tela požiadavky

Anotácia	Popis
@PathParam("x")	Parameter z URI – fragment {x}
@QueryParam("x")	Parameter dotazu – ?x=hodnota
@DefaultValue("v")	Náhradná hodnota, ak parameter chýba
@Singleton	Jedna inštancia triedy po celú dobu behu aplikácie
@Context	Injekcia kontextu: UriInfo, HttpHeaders, ServletContext

Tabuľka 6.2 – Anotácie knižnice JAX-RS

6.4 Implementácia triedy zdroja

```

@Path("/oznam") // URI: /resources/oznam
@Singleton
public class OznamResource {

    private String oznam = "Žiadny oznam.";

    @GET
    @Produces(MediaType.TEXT_PLAIN) // odpoveď: prostý text
    public String getOznam() {
        return oznam;
    }

    @PUT
    @Consumes(MediaType.TEXT_PLAIN) // telo: prostý text
    public void setOznam(String novy) {
        oznam = novy;
    }
}

```

6.5 Parametrizácia URI a trieda Response

```
// --- parameter z cesty URI ---
@GET
@Path("predmety/{kod}") // {kod} je premenná časť URI
@Produces(MediaType.APPLICATION_JSON)
public Response getPredmet(@PathParam("kod") String kod) {
    Predmet p = em.find(Predmet.class, kod);
    if (p == null)
        return Response
            .status(Response.Status.NOT_FOUND)
            .build();
    return Response.ok(p).build(); // 200 OK
}

// --- parameter dotazu s náhradnou hodnotou ---
@GET
@Path("predmety")
@Produces(MediaType.APPLICATION_JSON)
public List<Predmet> hladaj(
    @QueryParam("nazov") String nazov,
    @DefaultValue("20")
    @QueryParam("limit") int limit) {
    return em.createQuery(
        "SELECT p FROM Predmet p WHERE p.nazov LIKE :n",
        Predmet.class)
        .setParameter("n", "%" + nazov + "%")
        .setMaxResults(limit)
        .getResultList();
}
```

7 Integrácia REST a JPA

Kombinácia JPA a REST umožňuje vytvoriť plnohodnotnú dátovú vrstvu webovej aplikácie. JPA zabezpečuje perzistentné ukladanie entít; REST vystavuje prístup k nim prostredníctvom HTTP rozhrania.

7.1 Konfigurácia JPA pre webovú aplikáciu

```
<!-- persistence.xml – JTA transakcie spravuje aplikačný server -->
<persistence-unit name="myPU" transaction-type="JTA">
    <jta-data-source>jdbc/sample</jta-data-source>
</persistence-unit>
```

```
@Stateless // EJB – JTA transakcie automaticky
@Path("/produkty")
public class ProduktResource {

    @PersistenceContext(unitName = "myPU")
    private EntityManager em; // injektovaný kontajnerom

    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_XML)
    public Produkt nacistaj(@PathParam("id") long id) {
        return em.find(Produkt.class, id);
    }

    @POST
    @Consumes(MediaType.APPLICATION_XML)
    public Response vytvor(Produkt p) {
        em.persist(p);
        return Response.status(Response.Status.CREATED).build();
    }

    @PUT
    @Path("/{id}")
    @Consumes(MediaType.APPLICATION_XML)
    public Produkt aktualizuj(@PathParam("id") long id, Produkt p) {
        p.setId(id);
        return em.merge(p); // UPDATE v databáze
    }

    @DELETE
    @Path("/{id}")
    public void zmaz(@PathParam("id") long id) {
        Produkt p = em.find(Produkt.class, id);
        if (p != null) em.remove(p); // DELETE z databázy
    }
}
```

7.2 Serializácia do XML/JSON pomocou JAXB

```
@Entity
@XmlRootElement // povolí serializáciu do XML/JSON
public class Produkt implements Serializable {

    @Id
    @GeneratedValue
    private Long id;

    private String nazov;
    private BigDecimal cena;

    // gettery a settery
}
```

7.3 REST klient v Jave

```
Client client = ClientBuilder.newClient();
WebTarget target = client
    .target("http://localhost:8080/app/resources")
    .path("produkty");

// --- GET: deserializácia do objektu ---
Produkt p = target
    .path("42")
    .request(MediaType.APPLICATION_XML)
    .get(Produkt.class);

// --- POST: odoslanie objektu ---
Response r = target
    .request()
    .post(Entity.entity(p, MediaType.APPLICATION_XML));

// --- PUT: aktualizácia ---
target.path("42")
    .request()
    .put(Entity.entity(p, MediaType.APPLICATION_XML));

// --- DELETE ---
target.path("42")
    .request()
    .delete();
```

8 HATEOAS a pokročilé témy

8.1 Problém cyklických referencií

Obojsmerné asociácie medzi entitami vytvárajú cyklus referencií. Pokus o serializáciu do XML alebo JSON zlyhá výnimkou: *"A cycle is detected in the object graph."* Riešením je potlačenie serializácie jednej strany anotáciou **@XmlTransient**, pričom namiesto celých objektov sa serializujú len ich identifikátory:

```
@XmlAccessorType(XmlAccessType.PROPERTY) // serializujú sa getter metódy
@Entity
public class Obchod {

    @XmlTransient // zoznam kníh sa NEserializuje
    @ManyToMany
    private List<Kniha> ponuka;

    @XmlElementWrapper(name = "ponuka") // obaľujúci XML element
    @XmlElement(name = "isbn")
    public List<String> getIsbnList() { // virtuálna položka
        return ponuka.stream()
            .map(Kniha::getIsbn)
            .collect(Collectors.toList());
    }
}
```

8.2 Princíp HATEOAS

HATEOAS (Hypermedia as the Engine of Application State) je princíp, podľa ktorého REST odpoveď obsahuje **hypermediálne linky** na asociované zdroje namiesto ich priamej reprezentácie. Klient naviguje medzi zdrojmi výhradne prostredníctvom linkov obsiahnutých v odpovediach. Tento prístup rieši cyklické referencie a oddeľuje klienta od URI štruktúry servera.

```
@GET
@Path("/{nazov}")
@Produces(MediaType.APPLICATION_XML)
public Response getObchod(
    @Context UriInfo uriInfo,
    @PathParam("nazov") String nazov) {

    Obchod o = em.find(Obchod.class, nazov);
    if (o == null)
        return Response.status(Response.Status.NOT_FOUND).build();

    Response.ResponseBuilder rb = Response.ok(o);

    // pre každú knihu vloží link do HTTP hlavičky
    for (Kniha k : o.getPonuka()) {
        rb = rb.link(
            uriInfo.getBaseUriBuilder()
                .path("knihy")
                .path(k.getIsbn())
                .build(),
            "ponuka" // rel – identifikátor vzťahu
        );
    }
    return rb.build();
}
```

8.3 Klient so spracovaním HATEOAS linkov

```
Client client = ClientBuilder.newClient();

// krok 1: načítaj koreňový zdroj – obsahuje linky na obchody
Response resp = client
    .target(BASE_URI)
    .path("obchody")
    .request(MediaType.TEXT_PLAIN)
    .get(Response.class);

for (Link lo : resp.getLinks()) {

    // krok 2: načítaj detail obchodu
    Response r2 = client
        .target(lo)
        .request(MediaType.APPLICATION_XML)
        .get(Response.class);

    System.out.println(r2.readEntity(String.class));

    // krok 3: načítaj detail každej knihy cez linky v hlavičke
    for (Link lk : r2.getLinks()) {
        String kniha = client
            .target(lk)
            .request(MediaType.APPLICATION_XML)
            .get(String.class);
        System.out.println(kniha);
    }
}
```

9 Záver

Táto dokumentácia poskytla systematický prehľad technológií JPA a REST v kontexte vývoja Jakarta EE aplikácií. Boli predstavené základné koncepty ORM, pokročilá konfigurácia entitných tried a asociácií, stratégie mapovania hierarchií tried a kompletný postup návrhu RESTful webových služieb vrátane integrácie s perzistentnou vrstvou a implementácie štýlu HATEOAS.

Pre ďalšie štúdium odporúčame dokumentáciu špecifikácií Jakarta Persistence 3.2 a Jakarta RESTful Web Services 3.1, ako aj oficiálnu dokumentáciu implementácie EclipseLink. Všetky príklady boli overené s konfiguráciou: Java 21, EclipseLink 4.0.x, GlassFish 8.0, PostgreSQL 16.

10 Zoznam skratiek

Skratka	Plný výraz
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
EJB	Enterprise JavaBeans
FK	Foreign Key – cudzí kľúč
HATEOAS	Hypermedia as the Engine of Application State
HTTP	Hypertext Transfer Protocol
JAXB	Jakarta XML Binding
JAX-RS	Jakarta RESTful Web Services
JPA	Jakarta Persistence API
JPQL	Jakarta Persistence Query Language
JSON	JavaScript Object Notation
JTA	Jakarta Transaction API
MIME	Multipurpose Internet Mail Extensions
ORM	Object-Relational Mapping
PK	Primary Key – primárny kľúč
REST	Representational State Transfer
SQL	Structured Query Language
URI	Uniform Resource Identifier
WADL	Web Application Description Language
XML	Extensible Markup Language